

PLUG & CHUG

AI Agents, Explained

Why build your own local agentic system when you already have fancy apps like Codex, Claude Code, or WorkBuddy?



@kwis7

Reader edition 3 / 2026-09-22

github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide

In this guide

Start with your own AI assistant. Use this book to understand each step, and try the worked example when it helps.

Build an AI workspace you can return to	3
Before the first task: how an agent works	4
1. Finish one small task	11
2. Give the work a place to continue	13
3. Pick up where you left off	15
4. Keep a method that worked	16
5. Separate work when responsibilities collide	18
6. Use the workspace with another tool	20
7. Know what has actually been completed	21
8. Maintain what you use	23
9. Reference: the standard toolkit	24
Further reading	27

Build an AI workspace you can return to

You do not need to learn programming before using this guide. Begin with work you understand: reading, writing, studying, research, or a recurring task you would like help with. The aim is to build an AI working system that you understand and can use again tomorrow.

Good work should survive the end of a chat. Your sources, choices, useful methods, checked results and unfinished tasks need a home that the next session can find. Your existing AI app can help create that home and explain it as you go.

Start with your own assistant

Open the [beginner guide](#) and give its starting prompt to the AI app you already use. It includes the repository link. The assistant begins with a few questions about your work, app and computer, then proposes a small setup suited to you. You review where it will create files before it performs the technical steps its tools allow.

The first useful milestone is concrete: a local workspace, one checked result, a handoff, and a short card saying what to open and ask next time. Try that card in a new conversation. A proposed design, a created folder and a successful restart are different milestones; the assistant should show which ones actually happened.

This book explains the choices along that route. If a term is unfamiliar, ask the assistant to explain it using your own task. Terminal commands are an optional implementation route, collected in the final reference chapter and the [quick start](#).

Keep the work, change the model

The active model can change while the work remains yours. Your sources, decisions, procedures, task records and reviewed results should live in forms you can inspect and carry elsewhere. A change of interface or provider may require adaptation, but it need not erase what you have learned.

We call the configured environment around model execution a **harness**. It includes instructions, files, tools, task state and checks. Some parts are ordinary text; others are software that grants access or checks actions. We will distinguish them throughout the book.

The company metaphor helps: a model is the worker currently doing the reasoning, while the harness holds the working arrangements, records and tools. You remain the owner. It is a way to explain roles, not a claim that a model has human loyalty, awareness or automatic access to every file.

Learn through one example, then apply it to your work

Chapters 1–4 follow a small comparison of three fictional venues. The supplied notes let you see how to check a result, save it and recover the next step without introducing personal information. You may do this optional exercise, or use the same ideas with the task you chose during guided setup. You do not need a venue-comparison Agent to build a personal system.

Chapters 5–8 explain when more roles, a different tool, stronger checks or maintenance become useful. The reference chapter connects those ideas to the standard toolkit. Each part should answer a practical question that arises as your work grows.

The teaching workspace is deliberately small. The standard generator offers a fuller structure with task manifests and local checks. Your assistant should explain which approach fits your needs; a smaller teaching workspace is not an undocumented generator option.

Read evidence at its stated level

The example data is synthetic. Repository commands and compatibility records describe the included implementation; they do not certify an app installed on your computer. Instructions, static checks, fresh-session behavior and external delivery each need their own evidence.

Continue with [how an agent works](#) for the company diagram and core concepts. Keep your own task in mind while reading the example: what would make your next session easier to begin?

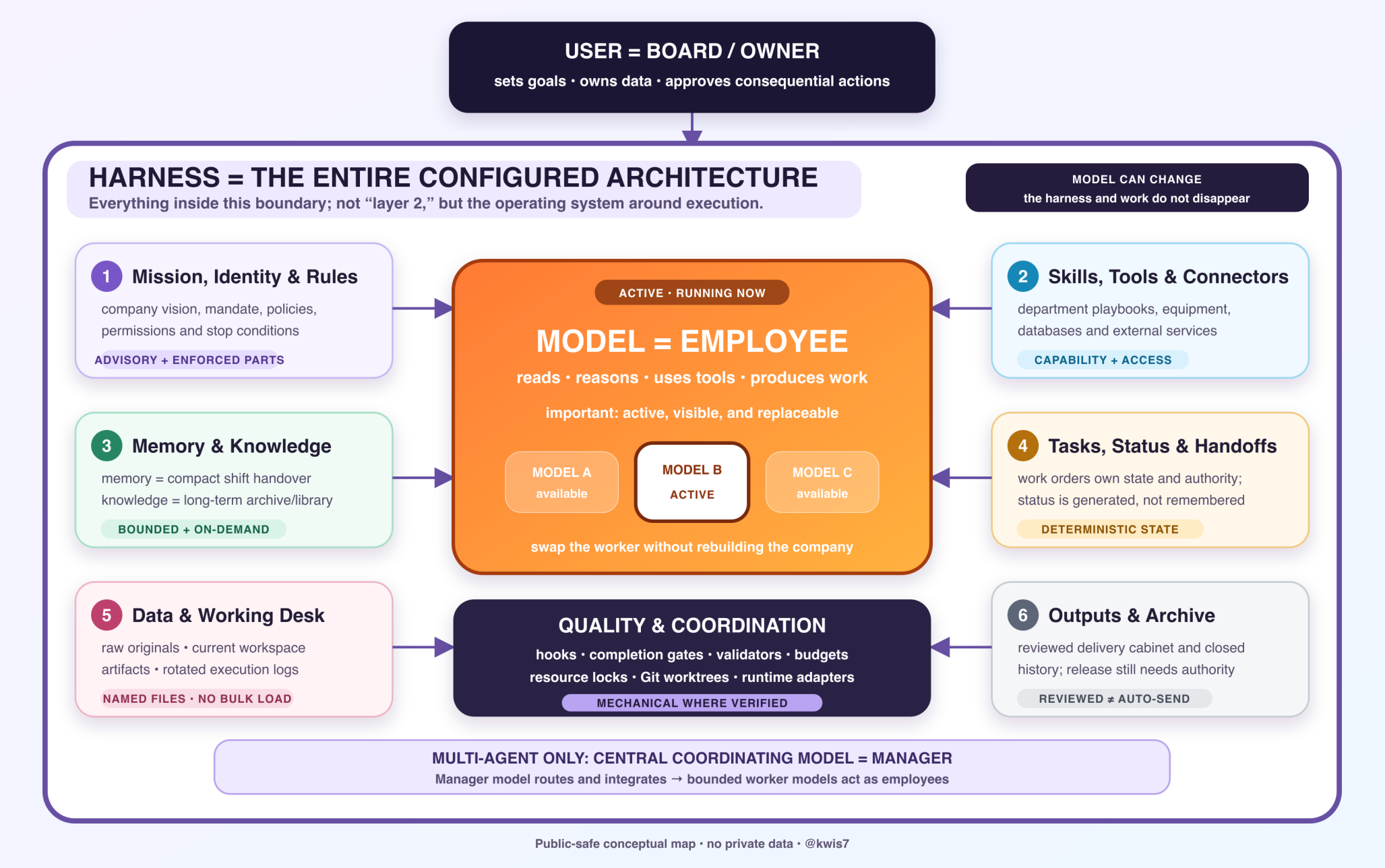
Before the first task: how an agent works

You ask an AI assistant to compare three sources. It reads files, writes a table, checks a missing detail, and saves a note for next time. Which part did the work: the model, the Agent, the prompt, or a Skill?

Each contributed something different. Understanding those contributions helps you build a useful system without mistaking a folder for a capability or a good prompt for a guarantee. Here, an **agent** means a role carried out through a model, instructions, working state, available tools, and an execution loop. The exact implementation varies. An agent can operate in one session without local files, and it does not require other agents.

Read the diagram as a working company

The diagram below presents the repository's complete design. It is a map of possible responsibilities, not a requirement to build every box before starting.



The active model inside the user's configured harness.

At the top, **you are the owner**: you choose goals, control your materials, and authorise consequential actions. In the middle, the **model is the active employee**: it interprets the material supplied to it, reasons about the request, and proposes a response or tool call. The model is not the company, its records, or its permissions.

An **Agent is a working role**, such as a research assistant responsible for comparing sources. Its instructions describe the assignment and boundaries; its state records progress; its tools provide available operations. The same model can fill several roles in separate sessions with different context and permissions. Different models can also fill the same role over time. A name such as “Research Agent” does not create a separately trained model.

The surrounding **harness** is the configured environment that makes this work possible: rules, files, procedures, tools, state, and checks. In the diagram, these arrangements and the active model occupy one system boundary. Replacing the model can preserve the surrounding work, although a new runtime may need different adapters and verification.

The company analogy helps distinguish responsibilities. Skills resemble playbooks, tools resemble equipment, memory resembles a short handover, and knowledge resembles a reference library. A coordinating model is a manager only when several roles need routing or integration. It is still a model in a bounded role, not a necessary extra layer or a higher source of authority.

None of this implies consciousness, loyalty, permanent memory, or automatic access. The diagram's quality controls are effective only where the corresponding software is implemented and verified. A written policy and a mechanically blocked action are different things.

Keep the core terms separate

Term	Its job in the system
Model	Interprets supplied context and generates responses or proposed actions
Agent	Carries a role through instructions, state, tools, and an execution loop
Prompt	Communicates a request, instruction, or other material to the model
Skill	Packages a reusable procedure and supporting resources
Tool	Performs an operation, such as reading a file or running a calculation
Runtime	Hosts execution, supplies context, dispatches tools, and handles results
Harness	Configures the working environment around model execution
File	Stores information that can persist beyond a session
Context	The material actually available to the model for the current response

These are design roles, not necessarily separate products. One application may provide execution, tools, storage, and an interface together.

A fixed workflow follows a predefined sequence. In an agentic loop, the model can choose subsequent steps in response to observations, within the system's limits. A practical system may combine both. This distinction follows [Anthropic's explanation of workflows and agents](#); it does not certify any particular product's implementation.

A prompt sets the task; a Skill carries a method

A task prompt might say: “Compare the supplied venues for 16 people after 18:00, with step-free entry required.” It tells the agent what you want now. Durable rules express instructions that should apply across a defined set of tasks, such as preserving original sources or keeping unsupported facts unknown. Both can influence the model when loaded, but they have different scope and expected lifetimes.

The model's input is not limited to your latest chat message. The runtime can assemble relevant instructions, loaded Skill contents, and tool observations into the current context as well.

At its core, a Skill in this repository is a reusable instruction document saved as `SKILL.md`. When loaded, its text becomes part of the model's current context. Think of it as **pinning a useful prompt or method** in a stable, discoverable place so you can reliably find and reuse it.

Pinning is an analogy for discovery, not permanent activation. In the [Agent Skills specification](#), the name and description support discovery first; the full instructions are loaded when needed. This does not mean the whole document appears in every turn or becomes a higher-priority system message. Actual discovery and loading depend on the host.

The document might teach requirement checks and source citation, with optional examples, references, or scripts alongside it. These materials supply a method, not new neural training or changed model weights. They do not grant tool permissions. An included script runs only through actual execution tools and their access rules; mentioning email does not connect an account or authorise sending.

The same model therefore has a clearer basis for comparison when it receives the task, source notes, and procedure than when it receives “pick a venue” alone. Better context helps shape its behavior, but the result still needs checking.

Follow one execution loop

The first exercise uses synthetic notes, not real venue information. Its request is a comparison, not a booking. A typical tool-using loop looks like this:

1. **Observe the request.** The runtime supplies the task and relevant instructions. The model identifies the requirements and which source files it needs.
2. **Reason about the next step.** If it cannot yet see the notes, the model proposes a file-read operation rather than treating the filenames as evidence.

3. **Request a tool.** The proposed call names an available operation and its arguments. A sentence saying “I read the file” is not a tool execution.
4. **Execute through the runtime.** The host applies its configured access policy and dispatches the allowed operation. It may return file contents, deny access, or report an error.
5. **Use the result.** Returned material enters the subsequent working context. The model compares the notes and may request another operation if more permitted evidence is needed.
6. **Review and save.** It drafts `workspace/comparison.md`. After source review and corrections, the checked result belongs at `outputs/comparison.md`, with actual progress recorded in `workspace/handoff.md`.
7. **Stop or hand off.** It reports the achieved result and remaining limits. Missing information or authority can require a pause instead of another action.

Cedar Hall's supplied note supports the recorded requirements. Willow Room's hours do not fit. Maple Studio's step-free entry is unspecified. The loop should preserve that unknown; reasoning cannot manufacture an entrance statement that the source lacks. Writing an unsent question is a possible next task, but it does not obtain the answer.

Failures can occur at every stage: a wrong path, unavailable tool, denied permission, incomplete tool result, or mistaken interpretation. Even a successful file write does not prove the file is correct. Without file tools, a chat interface can return text for you to save, but it must not claim automatic writeback. Chapter 1 turns this outline into a concrete exercise.

What Markdown files actually do

`.md` is a filename extension commonly used for **Markdown**, an editable plain-text format. A heading can begin with `#`; a list can use hyphens; links can point to other files. A person can read the underlying text without special software. Markdown is useful here because instructions, notes, and handoffs remain inspectable and easy to revise.

Some names have meaning only through a host's conventions. `AGENTS.md` or `SKILL.md` can serve as instruction or skill entrypoints when the selected environment discovers and loads them in the expected location. They are not universally executable files. Names such as `RULES.md` and `workspace/handoff.md` express project conventions; the workflow must arrange for the relevant content to be read.

These tiny excerpts illustrate different responsibilities. They are not additional required files for the first exercise or a complete installable Skill:

```
task-brief.md
Compare the supplied venues for 16 people after 18:00.
Step-free entry is required.

RULES.md (illustrative project rule)
Keep missing source facts unknown; do not invent them.

skills/source-comparison/SKILL.md (illustrative procedure)
Check each requirement against each source and cite the evidence.

workspace/handoff.md (example state, only if actually true)
Draft saved at workspace/comparison.md; source review remains.
```

Three separate things now exist: **stored files**, **current context**, and **model weights**. Files retain text. Context contains the selected instructions, conversation, and tool results supplied for the current work. Weights are the model's learned parameters. Saving or loading these files does not by itself update those parameters. A future session resumes by retrieving relevant records, not because every saved fact has become part of the model.

Personalise the work explicitly

A personal system should fit recurring work you actually do. You can specify language, detail level, evidence standards, review points, and preferred deliverables. These choices become useful when their scope is clear and you can revise them.

Suppose you correct a comparison: “For future source comparisons, lead with a short recommendation, then explain the evidence and unknowns.” That explicit preference can become a rule for comparison work. It need not apply to a poem, a debugging log, or every future conversation. Record what you requested, not an inferred personality profile.

Other lessons belong elsewhere. “Maple's access remains unknown” is current task state. The repeated sequence of extracting, comparing, citing, and reviewing can become a Skill. An explanation of why a missing field is not a negative answer can become a knowledge note. The distinction prevents one oversized memory file from collecting preferences, unfinished tasks, procedures, and reference material indiscriminately.

Personalisation changes the information and procedures available to the model. It is not automatic training, and repeated use does not prove a guessed preference. Sensitive traits should not be inferred from ordinary corrections. Retain only appropriate, authorised material, and review whether older preferences still apply.

Why build this alongside existing AI software?

If you already use Codex, Claude Code, or WorkBuddy, the point is to organise your work within the software you choose. This guide does not assume those products lack memory, Skills, or workspace features. Reuse the runtime, tools, instruction loading, and storage features available in your chosen product. You do not need to write a new orchestration engine or custom application.

Your personal system is the deliberate combination of a stable role, your own authorised sources, scoped preferences, repeatable methods, and current state with evidence. Software supplies capabilities; you decide how they support your work. Some people will find the built-in arrangements sufficient as they are.

Here, **local** means keeping a durable workspace under your control. Files can be inspected, backed up, and, where appropriate, versioned to show what changed. Sources stay connected to conclusions. Another session can recover the next step, and another compatible tool can reuse the authorised materials.

Local files do not mean the model runs locally or that the workflow is offline. A hosted model or connector may receive content when used. Privacy depends on actual data flows, access permissions, service settings, and what you provide—not on the .md extension or folder name. Portability likewise requires checking how the next tool loads instructions and saves results; tools are not universally interchangeable.

This approach pays off when work repeats, spans sessions, or needs a traceable result. For a one-off explanation, normal chat may be enough. A durable system also costs effort: instructions can become stale, copies can diverge, and procedures need review. Start with one useful task and retain what makes its next occurrence easier. The [first-task chapter](#) shows that smallest working step.

1. Finish one small task

This chapter uses the optional fictional exercise to make the method visible. If you have already chosen your own task in the [beginner guide](#), use the same sequence—agree on the result, work from named inputs, check, save and leave a handoff—with that task.

Suppose you are planning a workshop for 16 people after 18:00. Everyone must be able to enter without steps. Before anyone contacts a venue, you want a short comparison that distinguishes a suitable candidate from an unsuitable or uncertain one.

The [first-project exercise](#) contains three synthetic source notes. They represent fictional venues, not real businesses or verified live information. The assignment is a source-comparison exercise; it grants no permission to book, send messages, or make payments.

Know the result you want

A useful answer is more than a ranking. It should show the recorded facts, apply each requirement consistently, explain the resulting judgment, and preserve what remains unknown.

Venue	Capacity	Recorded hours	Step-free entry	Judgment
Cedar Hall	18	18:00–21:00	Yes	Meets the recorded requirements
Willow Room	24	09:00–17:00	Yes	Fails the evening-hours requirement
Maple Studio	20	18:00–22:00	Unspecified	Cannot confirm accessibility

Cedar Hall is the suitable candidate on the supplied facts. That is a bounded conclusion. It does not establish availability on a particular date, the duration of a workshop, price, or booking terms. Maple Studio's missing accessibility statement is an unknown; it is not evidence that the venue is inaccessible. Willow Room's larger capacity does not compensate for its hours.

Choose an available working route

Use an AI tool you already have. For direct file work, it must be able to read the exercise files and write inside your chosen working copy. Check its actual permissions before starting. If your interface only accepts pasted text or uploads, provide the three notes explicitly and save the returned Markdown yourself. That route still teaches the method, although filesystem loading and writeback remain manual. Your existing model or API usage may carry its normal costs; the exercise itself needs no paid data service.

Copy the exercise into a local practice folder if you want to preserve the bundled reference files. Tell the agent exactly which copy it may change. From the exercise root, a suitable assignment is:

```
Read task-brief.md, sources/cedar.md, sources/willow.md,  
and sources/maple.md. Do not read expected/ yet.  
Compare the fictional venues for 16 people after 18:00,  
with step-free entry required. Use only the supplied notes.  
Write workspace/comparison.md with source links, judgments,  
and unknowns. Identify missing inputs instead of guessing.  
Do not browse, contact, or book. Agree on any overwrite first.
```

The tutorial agent first creates the draft. In a chat-only interface, you perform that saving step and mark local file loading untested. The repository's `expected/comparison.md` and `expected/handoff.md` are reference answers, not proof that your own exercise has been completed. Leave them closed until you have attempted the draft.

Review before accepting

Open your `workspace/comparison.md` and follow its links to the three files under `sources/`. Confirm the capacities and time windows yourself. Then inspect the reasoning: all three have enough capacity, only two have evening hours, and only Cedar has both evening hours and a recorded step-free entrance. You can now compare it with `expected/comparison.md`.

If the answer says Maple is suitable, ask which source establishes its accessibility. The correct repair is to mark that requirement unknown, revise the judgment, and record the correction. A polished paragraph cannot fill a missing source field.

After review and any corrections, ask the agent to save the checked version at `outputs/comparison.md`, preserving the draft. Then have it create `workspace/handoff.md` with the actual checks, remaining unknowns, permitted inputs and writes, and next step: an unsent clarification question about Maple's entrance. Record recovery as untested until you observe it. A real booking decision still requires new information and separate authority.

2. Give the work a place to continue

A finished comparison preserves an answer. A useful workspace also preserves how to understand that answer: the inputs, the requirements, the checks, and any unfinished work. Without those, the next conversation can easily repeat the research or inherit a conclusion whose limits have been forgotten.

Our example needs only a small folder. The supplied source notes stay separate from the working result and handoff. The bundled `expected/` directory remains a teaching reference. Nothing about this layout requires a new agent, a database, or the full scaffold.

Give each file a clear job

The source notes establish what was supplied. `workspace/comparison.md` is the draft; after review, `outputs/comparison.md` becomes the checked answer. `workspace/handoff.md` records current state and the next permitted step. These files have different jobs: the draft is not a second authority for the reviewed result. If a decision changes, review the change, update the result, and record its status in the handoff.

As work grows, these distinctions become useful:

Kind of information	Question it answers	Example
Source material	What evidence was supplied?	Cedar Hall's recorded hours
Task state	What is happening now?	Comparison reviewed; no venue contacted
Memory	What short pointer helps us resume?	Location of the active task and key decision
Knowledge	What stable understanding is reusable?	How missing evidence affects a comparison
Skill	What procedure should we repeat?	Compare each candidate against every requirement

Context is different: it is the material actually supplied to the model in the current session. A file can exist on disk without entering context. An instruction to read a source is also different from evidence that it was read. Ask the agent to identify the files and facts supporting its answer when loading matters.

Preserve sources and uncertainty

Do not rewrite Maple's source note to make the comparison easier. Leave “accessibility unspecified” in the input and explain its consequence in the result. If new evidence arrives later, preserve its source and date, then revise the affected judgment. This keeps changes traceable without retaining every sentence of the conversation.

The same principle applies to real work. A summary is an interpretation of a source, not a replacement for provenance. Separate the source's claim from your conclusion, and retain enough attribution for a reviewer to inspect the bridge between them. Store only material you are authorised to retain and make available to the tool.

Let structure follow need

The standard scaffold gives larger projects named places: `raw_data/` for original inputs, `workspace/` for drafts, `artifacts/` for generated intermediates, `logs/` for execution traces, `outputs/` for reviewed results, and `archive/` for closed material. Their full responsibilities are defined in the [filesystem contract](#).

Those names express intent. They do not enforce permissions. A tool that can read the whole directory may still reach a file called `private`. Sensitive work therefore needs actual access restrictions and a deliberate choice about which material enters the model. A useful public example should contain synthetic or authorised material, not a disguised copy of someone's private records.

Before adding folders, try one practical test: can you identify the current answer, its sources, and the next action within a minute? If yes, your present structure may be enough. If not, fix the ambiguity that caused the delay. Structure earns its place by reducing the cost of continuing.

3. Pick up where you left off

Continuity is something you can test. Close the original conversation, start a fresh session, and see whether the saved files support the next step. This is more informative than asking the original model whether it will remember.

For the venue exercise, the task is already useful as a comparison. The handoff should prevent a later session from treating it as a completed booking or silently converting Maple's unknown into a positive answer.

Leave a handoff with a next action

A handoff is a short working note, not a transcript. It should tell a newcomer what the task was, what exists, why the result stands, and what would change it. For example:

```
Task: compare three fictional venues for 16 people after 18:00;
step-free entry is required.
Result: outputs/comparison.md. Draft: workspace/comparison.md.
Sources: sources/cedar.md,
sources/willow.md, sources/maple.md.
Checked: capacities, hours, and accessibility statements.
Decision: Cedar meets the recorded requirements; Willow fails
the hours requirement; Maple's accessibility remains unknown.
Limit: synthetic notes only. Availability, price, and exact event
duration are unverified. No contact or booking is authorised.
Next: verify the reviewed result exists, then draft one unsent
question about Maple's step-free entrance in workspace/maple-question.md.
Read only this handoff, task-brief.md, the reviewed result,
and the three named sources. Write only the question and this handoff.
Recovery: not tested until a fresh session is observed.
```

This note preserves a decision and its limits without preserving all the drafting conversation. The reference handoff in [expected/handoff.md](#) is a model to compare against, not a replacement for recording what your own session actually checked.

Run the restart exercise

Open a fresh session in the practice copy, or explicitly supply its handoff and named sources. Ask it to read `workspace/handoff.md`, inspect only the permitted files, and state the current result and unresolved issues. Then carry out the recorded next step: save one unsent question about Maple's step-free entrance at `workspace/maple-question.md` and update the handoff. No browsing, contact, or booking is allowed.

A successful restart finds the reviewed result, identifies Cedar as the supported candidate, preserves Maple's unknown, and recognises Willow's hours problem. Reopen the new question and updated handoff before recording recovery as tested with the tool, date, and observed outcome. Drafting a question does not answer it. If a named input is absent, the session should report that gap rather than reconstruct missing evidence from memory.

To practise failure handling safely, follow the tutorial's missing-source prompt in another fresh session. Permit only `task-brief.md`, `sources/cedar.md`, and `sources/willow.md`; declare Maple's note unavailable without deleting it. Exclude other exercise files and remembered facts. Save the limited comparison at `workspace/missing-source-check.md`, leaving Maple unassessed and the reviewed result untouched. This tests a declared input boundary; mechanical isolation depends on the tool's permissions.

Recover before repeating

Interruptions often leave partial work. Before retrying, inspect the draft, reviewed result, and `workspace/handoff.md` to identify the last successful step. A missing reviewed result sends you back to review; a partial draft is not a checked output. Repeating a read or regenerating a disposable draft is usually straightforward. Repeating an external action can create a duplicate booking, message, or payment.

When an external operation has an uncertain result, first check the external record. A timeout means the response was not received; it does not prove that nothing happened. The fictional exercise has no external actions, but practising the distinction makes later workflows safer.

For larger work, the toolkit records state in `tasks/<task-id>/task.yaml` and generates `STATUS.md` from it. That adds explicit scope, verification, and lifecycle fields. The underlying habit remains the same: record current truth once, preserve useful partial work, and leave the next action precise enough that another session can continue without guessing.

4. Keep a method that worked

After several comparisons, you may notice that the valuable part is the procedure. You identify the requirements, extract the same fields from each source, apply the requirements consistently, and expose missing evidence. That method can survive even when the venues, documents, or active model change.

A **Skill starts with a Markdown instruction document**, conventionally named `SKILL.md`. Its text becomes part of the model's context when the host loads it. You are keeping a useful prompt or method in a discoverable place—much like pinning a playbook for reuse. The model follows those instructions with the capabilities it already has; saving a Skill does not train new model weights.

That pin is a way to find the method again, not a promise to include its full text in every turn or give it higher instruction priority. The name and description help the host choose it; the full procedure and any supporting references are loaded as needed. A Skill can also include examples, templates, or scripts, but scripts still need actual tools and permission to run. It earns its maintenance cost when repetition or a recurring mistake makes the procedure worth preserving.

Extract the method, leave the case behind

“Choose Cedar Hall” is a conclusion from one fictional exercise. “Check every candidate against every required condition” is a reusable method. Mixing the two can cause the next comparison to inherit an old answer.

For our example, a useful skill might be described as follows:

```
Name: Source-backed comparison
Use when: comparing supplied candidates against explicit needs.
Inputs: requirements and named source notes.
Method: extract facts with sources; check each requirement;
        separate meets, fails, and unknown; explain the final judgment.
Output: concise comparison, unresolved questions, and handoff.
Limits: do not infer missing facts or perform external actions.
```

This is a teaching sketch, not a universally installable skill format. The repository's [skill template](#) shows its packaged form. Actual discovery depends on the selected runtime and installation location.

Write a useful trigger

The description should help select the procedure before its full contents are loaded. “Helps with research” is too broad. It gives no reason to choose this skill over summarisation, writing, or data collection. A useful description names the work, expected inputs and outputs, and exclusions.

Try a small set of routing examples. “Compare these three supplier notes against our requirements” should activate the comparison method. “Write a thank-you note” should not.

“Compare these venues and book the best one” contains two different actions: the skill can prepare the comparison, while booking needs separate capability and authority. A trigger must not quietly authorise the second action.

Then test the procedure itself. The venue fixture should produce Cedar as supported, Willow as unsuitable on hours, and Maple as unresolved on accessibility. Add a variation with a missing source or contradictory fact. These tests reveal whether the skill handles uncertainty, not merely whether it repeats the reference answer.

Put each lesson in the right place

A preference that should always apply belongs in the relevant operating rules. Stable background belongs in knowledge. Today's unfinished comparison belongs in task state. A repeated sequence of work belongs in a skill. A tool belongs where real execution or external access is needed.

This separation keeps skills small enough to understand. The short description supports discovery; the main instructions explain the method; deeper references are loaded when the task needs them. Loading every skill and reference into every conversation defeats that design and consumes attention without improving the current answer.

Share procedure deliberately

Another agent can borrow the comparison method without receiving private candidate notes, internal decisions, or a full chat history. Share the procedure and a synthetic example, then let the receiving project supply its own authorised inputs. This is how useful learning travels while data ownership stays clear.

Review a new skill after real use. Keep a correction when it addresses a detectable failure, and avoid turning every unusual request into a permanent rule. The deeper [distillation guide](#) explains how to promote repeated experience without filling the system with overlapping procedures.

5. Separate work when responsibilities collide

One agent with a few well-chosen skills can handle a substantial amount of work. A new subject or a different writing format does not automatically require another agent. Splitting becomes useful when recurring work has a distinct mission, data boundary, source base, authority, or lifecycle.

Think about the failure you are trying to prevent. If source analysis keeps changing while a designer is preparing the final report, separate ownership may help. If the only problem is an inconsistent table format, a template or skill is probably enough.

Choose the right unit

Need	Usually start with
One temporary objective	A task or project
A procedure repeated across tasks	A skill
Stable reusable reference material	A knowledge collection
Recurring work with distinct ownership	A domain agent
One bounded independent contribution	A temporary subagent
Access to a service or executable action	A tool or connector

A domain agent is a durable responsibility expressed through its instructions, working files, and access arrangements. It need not use a permanently assigned model. Choosing a different model changes the active reasoning component; it should not silently change who owns the sources or which actions are allowed.

For the venue exercise, one agent is sufficient. In a larger recurring event workflow, a research owner could maintain source comparisons while a communications owner prepares approved messages. Researching an option and contacting it have different authority requirements. Separating them can make that boundary easier to review, but it still needs tool permissions and explicit action rules.

Define ownership before delegation

A useful delegation says what result is needed, which inputs may be read, where the worker may write, what is forbidden, and how the result will be checked. “Research this” leaves too much room for conflicting assumptions.

For example, a temporary reviewer could inspect the three fictional notes and report any unsupported claim in `workspace/comparison.md`. It needs read access to those files and a place for its review. It does not need to rewrite the sources, alter the task's scope, or contact venues. The lead retains responsibility for integrating the findings and checking the final answer before saving it to `outputs/comparison.md`.

Independent review is especially valuable when a plausible mistake could survive self-review. It costs additional time and context, so use it for a reason. A second agent repeating the same assumptions without checking the sources adds little confidence.

Keep one writer per authority

Two agents can read the same source. Two agents editing the same current-state file can erase each other's work. Assign one writer to every authoritative path and let other contributors produce separate proposals or artifacts.

Git worktrees can isolate concurrent code changes in separate checkouts. Resource locks coordinate access to a shared file, device, port, or other resource. They solve different problems: a worktree does not isolate a shared account, and a lock is not a privacy control. Neither grants permission to publish or perform an external action.

A control center becomes useful when several durable owners need routing or coordinated work. Its job is to know where responsibilities belong, manage dependencies, and integrate results. That does not make it the owner of every domain's private facts. Share methods across agents; transfer data only within its authorised scope.

Test the boundary in ordinary language

Write two requests that clearly belong to an agent, two that do not, and one that overlaps with a neighbour. Check whether the descriptions route them sensibly. The [routing guide](#) provides a fuller method.

If you cannot explain why a new agent needs its own responsibility, keep the work with its current owner. Add separation when it makes a real boundary clearer, and retain a single accountable lead for each result.

6. Use the workspace with another tool

Portable work remains understandable outside the conversation that created it. The venue notes, comparison, handoff, and reusable method can all be read by a person or supplied to another model. That is the foundation of portability. Automatic loading, tool access, and enforcement require an additional layer of adaptation.

Changing a model and changing a runtime are different operations. The **model** performs the reasoning. The **provider** supplies inference through a service or backend. The **runtime** hosts the session, assembles context, exposes tools, and may manage permissions or hooks. A **switchboard** can route model or provider configuration without owning your task records or working files.

Identify what actually needs to move

Start with the smallest useful package: the task's requirements, authorised sources, current result, handoff, and relevant procedure. Include the operating rules that affect the task. Do not upload an entire personal workspace simply because the destination accepts many files.

Then ask how the destination receives this material. A local tool may discover designated entrypoints. A hosted workspace may require project instructions and selected uploads. A custom API application must assemble context and implement persistence itself. A document saying “read my files” cannot create filesystem access in an interface that lacks it.

Also decide where changes return. If the new tool creates a revised comparison in chat, someone must save it to the authoritative workspace. Two copies with no reconciliation rule can diverge even when both models reason correctly. Manual writeback is a valid workflow when it is explicit.

Read support as evidence, not a promise

The repository's [compatibility manifest](#) records product categories, generated configuration, limitations, and evidence status. Its recorded review date is 2026-08-09. In that record, Codex, Claude Code, and Gemini CLI are classified as `verified_static`, with fresh-session verification marked `not_run`. This is a statement about the repository's evidence, not a new certification of current vendor behavior.

Other entries distinguish documented integrations, manual projections, provisional support, providers, switchboards, and custom-harness patterns. An adapter file is useful guidance, but its existence is not proof that a product loaded it or enforced a rule. Read the [adapter selection guide](#), then the adapter for the tool you intend to use. Recheck the product's official instructions when installing or changing an integration.

Test the destination with a small fixture

Use the synthetic venue exercise before migrating meaningful work. In a clean destination session, verify four separate behaviors:

1. It can identify the intended rules and named source files.

2. It reproduces the comparison without inventing Maple's accessibility.
3. It saves the result and handoff where the workflow expects them, or clearly leaves saving to you.
4. If a mechanical gate is claimed, an intentionally invalid completion is blocked and a valid completion passes.

Record the tool version, date, fixture, observed result, and remaining gaps. A model's statement that it “understands the rules” does not establish gate behavior. Conversely, a missing gate does not make a manual comparison impossible; it changes what you can claim and what you must review yourself.

Keep the original working copy until the new route has been checked. The aim is to carry forward your evidence and methods while adapting the parts that depend on a particular runtime. Portability becomes practical when both the durable files and the migration limits remain visible.

7. Know what has actually been completed

“Done” can mean a file was created, a comparison was reviewed, a test passed, or an external action succeeded. Those claims need different evidence. A reliable workflow names the intended result before work begins and checks the evidence that corresponds to it.

For the venue exercise, completion means the comparison matches the supplied notes, the judgments follow the stated requirements, unknowns remain visible, and a usable handoff exists. It does not mean the workshop has a confirmed location. That distinction should survive both the final reply and the next session.

Match the check to the claim

Claim	Evidence to look for
A result was saved	Open the exact file and inspect its contents
A comparison is supported	Trace material facts and judgments to the sources
A script passed	Record its command, exit status, and relevant output
A document is usable	Render it, inspect pages, and test navigation
A runtime loaded instructions	Observe a fresh session using the intended endpoint
A completion gate works	Observe invalid and valid cases in that runtime
An external action succeeded	Check the service's authoritative result or readback

A **receipt** records what actually happened: the check, date, inputs, relevant tool version, result, and where its evidence lives. A planned command in a task file is not a receipt. For a small comparison, a short review note can be enough. Higher-consequence work may need test logs, hashes, screenshots, or a separate reviewer.

Automated checks are particularly good at explicit conditions such as a missing file or stale status page. They may not detect that a fluent conclusion misinterprets a source. In our example, checking that the word “accessibility” appears is weaker than checking whether Maple's accessibility was treated as unknown.

Separate instructions from enforcement

An instruction can tell the agent to stop before booking. A restricted tool policy can prevent it from accessing a booking action. A validator can reject a task record with missing evidence. A runtime hook can act on that validator's result if the integration is actually loaded and behaves as expected.

These mechanisms complement one another, but they are not interchangeable. The repository's closeout gate checks structural conditions such as output presence, declared verification state, receipt paths, current status, budgets, and released locks. It does not independently determine whether every claim in a report is true. A dishonest or mistaken “passed” record still needs substantive review.

The runtime must also connect the gate correctly. A script returning an error in a terminal does not prove that the host blocks a final completion claim. That requires a fresh-session test with an intentionally incomplete task, followed by a valid one. Keep static and runtime evidence separate.

Make partial completion useful

If Maple's source cannot be read, report which parts were checked and which remain unverified. If the task only requires an evidence-bounded comparison, “unknown accessibility” may be a correct final result. If it requires confirmed accessibility for every option, the same gap prevents completion. The task contract decides the difference.

When work cannot continue, preserve the usable files, explain the dependency, and name the next action. Use an honest failed or cancelled outcome when appropriate. Do not invent successful receipts to satisfy a gate or tidy a dashboard.

Good verification reduces ambiguity for the next person. It makes clear what they can rely on, what they should inspect, and which new evidence could change the result. The [reliability reference](#) expands the task, receipt, gate, and recovery mechanics.

8. Maintain what you use

A workspace should become easier to resume as it accumulates useful work. That does not happen by saving everything. Unreviewed transcripts, duplicate procedures, stale dashboards, and obsolete instructions can make the next session slower and less reliable.

Maintenance begins with a practical question: which retained information helps a future task, and where should that information live? Preserve decisions, provenance, useful methods, and recovery pointers. Move or retire material when its current location no longer serves that purpose.

Close the task cleanly

After the venue comparison, retain the named sources, reviewed answer, relevant verification note, and handoff. There is little reason to place every drafting attempt in compact memory. If a correction revealed a reusable lesson, consider whether an existing skill or knowledge note should capture it.

Not every task requires a memory edit. Memory is a short recovery aid. It is useful when a durable decision or pointer would otherwise be hard to rediscover. Task-specific progress belongs with the task, and extensive explanation belongs in knowledge or a referenced artifact.

In the standard scaffold, update the task record and regenerate `STATUS.md`. Do not rewrite status independently in several places. When the work is closed, make the final outcome and remaining limits clear enough that a later reader will not mistake an old draft for an active assignment.

Review friction, not appearances

Choose a review interval that reflects actual use. A frequently used system may benefit from a brief weekly check; a quiet project may need attention only when reopened. A schedule written in a document is a suggestion until an actual scheduler is configured and tested.

Look for evidence of recurring friction: repeated explanations, lost sources, ambiguous routing, oversized startup files, skipped verification, or contradictory copies. Fix the smallest cause. A confusing skill description may need one sentence; it does not necessarily require another agent or a new framework.

Keep a short before-and-after record for substantial changes. For example: “New sessions were reading the entire archive; the entrypoint now points only to the active task and retrieves older notes when relevant.” Test that behavior in a fresh session. Editing the instruction alone establishes only that the text changed.

Keep startup context bounded

The toolkit defines budgets for instruction files, memory, task records, and large-file indexing. These limits encourage selective loading. They do not imply that filling every file to its limit improves performance.

When a file becomes too large, preserve meaning before shortening it. Move stable explanations to knowledge, task details to their task, and closed history to an archive. Keep concise pointers at startup. Never remove the only record of an unresolved decision merely to make a size check pass.

Large inputs should have a manifest that records their path, size, purpose, and loading policy. The model can inspect that index and request the relevant portion instead of reading an entire warehouse of data. Actual access controls must still protect restricted material; an index is not a permission system.

Change structure reversibly

Before merging agents, retiring a skill, or moving authoritative files, identify the owner, references, active tasks, and recovery path. Preserve a recoverable version where appropriate and obtain the required authority for destructive or external actions. A clean directory tree is less valuable than understandable history.

The [review and renewal guide](#) offers a deeper routine. Use it when you have enough real work to evaluate. The best maintenance outcome is often a smaller system whose remaining parts have clear jobs and demonstrated value.

9. Reference: the standard toolkit

The first-project folder teaches a small working method. The Portable Agentic System toolkit implements a broader standard scaffold with domain folders, task contracts, generated status, budgets, locks, and adapter configuration. It is an alternative setup route when those features are useful; completing the learning exercise does not require generating it.

The command examples below run from this repository's root. They use local Python scripts and assume python3 is installed and compatible with the source. Inspect the scripts, their `--help`, the proposed target, and your environment before execution. Use a new disposable directory for a first scaffold. These instructions do not claim that the commands have run on your machine.

Preview and create a scaffold

Read the [starter configuration](#) and copy it to a working file if you want to change its sample domains. The bundled file contains two illustrative owners; it is not a universal recommendation. Each configured agent needs a purpose, routing description, exclusions, and examples that distinguish it from adjacent work.

Choose an unused target path and run a preview:

```
python3 skills/portable-agentic-system/scripts/create_agentic_system.py \
--root /tmp/pas-learning-demo \
--config skills/portable-agentic-system/pas/examples/starter-config.json \
--dry-run
```

The preview reports the target root and agent paths as JSON; it does not enumerate every file to be written. Review those paths, the templates, and the generator's scaffold behavior before creation. To create the reviewed scaffold, run the same command without `--dry-run`. The generator refuses existing-file collisions by default. Do not reach for `--force` as a routine error fix; inspect the collision and preserve existing work.

The generated system includes runtime configuration and local operational scripts under `.pas/bin/`. Review hooks before trusting or enabling them in your chosen tool. The generator does not itself prove that a runtime has discovered or executed the configuration.

Run static checks

After generation, these commands inspect the scaffold. Replace the target consistently if you chose a different path.

```
python3 skills/portable-agentic-system/scripts/validate_agentic_system.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/check_budgets.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/check_descriptions.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/generate_status.py \
/tmp/pas-learning-demo --check
python3 skills/portable-agentic-system/scripts/harness_health_check.py \
/tmp/pas-learning-demo --json
```

Read every failure before continuing. If task records changed and status is stale, run `generate_status.py` with the same root but without `--check`, then check again. That regeneration writes `STATUS.md`; check mode only verifies consistency.

The adapter probe is also available:

```
python3 skills/portable-agentic-system/scripts/adapter_smoke.py \
/tmp/pas-learning-demo --runtime codex
```

This is a **static** probe despite its name. Its reported `verification_level` is `static`. Select a runtime matching your intended integration and follow its adapter for fresh-session tests. Do not upgrade the claim based on a passing configuration inspection.

Understand task authority

A standard `tasks/<task-id>/task.yaml` records identity and ownership; objective and scope; allowed inputs, writes, tools, and prohibited actions; outputs and completion conditions; verification and receipts; execution resources; and handoff. The generator writes JSON syntax inside the YAML-named file, which its standard-library parser can read. The parser also accepts a conservative YAML subset; it is not a promise of full YAML support.

Use the generated task as a schema example and consult the [governance reference](#) before constructing one manually. A teaching handoff from chapter 3 is not a complete valid task manifest. For a real task already recorded in the generated system, invoke its bundled gate from that system's root:

```
python3 .pas/bin/closeout_gate.py . tasks/T-123/task.yaml
```

Replace T-123 with an existing task. A successful terminal closeout requires the declared outputs, verification state and receipt paths, current generated status, budget compliance, released task locks, and handoff. Failed and cancelled tasks use honest unsuccessful outcomes. The gate's structural checks do not replace a substantive review of the deliverable or verification of the host's blocking behavior.

Find deeper implementation detail

Topic	Maintained reference
File authority and loading	Filesystem contract
Private material and external actions	Privacy boundaries
Routing and descriptions	Routing evaluations
Runtime classification	Compatibility manifest
Adapter selection and testing	Adapters
Tasks, gates, and concurrent work	Reliability reference
Full design interview	Build workbook

The repository's Python test suite runs with `python3 -m unittest discover -s tests -v`. Passing it supports the tested local behavior. It does not establish fresh-session compatibility, publication, delivery, or any external transaction. Report each of those only at the level of evidence actually obtained.

Further reading

Links lead to this repository and its technical references. Product guidance carries its own evidence date; a link does not establish a new runtime verification.

beginner guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/docs/start-here.md>

quick start

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/QUICKSTART.md>

Anthropic's explanation of workflows and agents

<https://www.anthropic.com/engineering/building-effective-agents>

Agent Skills specification

<https://agentskills.io/specification>

first-project exercise

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/examples/first-project/README.md>

filesystem contract

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/filesystem-contract.md>

expected/handoff.md

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/examples/first-project/expected/handoff.md>

skill template

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/templates/skill/SKILL.md>

distillation guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/skill-distillation-and-fusion.md>

routing guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/description-routing-evals.md>

compatibility manifest

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/compatibility/runtime-compatibility.json>

adapter selection guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/adapters.md>

reliability reference

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/textbook-reliability.md>

review and renewal guide

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/system-review-and-renewal.md>

starter configuration

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/examples/starter-config.json>

governance reference

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/docs/reference/task-lifecycle.md>

Privacy boundaries

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/privacy-boundaries.md>

Build workbook

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/textbook-build-workbook.md>